現在讓我用六人對談的格式，把這份實作中最深刻的設計決策攤開來講。

第一步：問題擴充與重構

多維度拆解

你的原始問題「用 DSPy + GEPA 實現七層 IR」看起來是一個實作問題，但實際上至少包含五個正交的子問題：

子題 A：映射問題——DSPy 的 Signature/Module/Optimizer 三件套到底對應 IR 中的什麼？Signature 是指令宣告？Module 是 Composite？Optimizer 是 compilation pass？這個映射必須在寫第一行程式碼之前定義清楚，否則會把抽象層次搞混。

子題 B：確定性-統計性混合問題——七層中前三層完全確定性（純 Python 函式），後四層需要 LLM。DSPy 原生假設所有模組都是 LLM 驅動的。怎麼在同一個 `dspy.Module.forward()` 裡混合「不用 LLM 的 numpy 計算」和「用 LLM 的 ChainOfThought 呼叫」？

子題 C：GEPA 回饋設計問題——GEPA 的核心能力是利用文字回饋（不只是純量分數）來反思和改進 prompt。聲學領域的物理約束要怎麼編碼成 GEPA 能理解的文字回饋？這是一個「物理知識→自然語言→prompt 優化」的翻譯鏈。

子題 D：多速率執行問題——真實助聽器要 <50ms 延遲，不可能每幀都跑七層。Harness 的 Scheduler 怎麼在 DSPy 框架裡實現「快速通道用 cached 參數，慢速通道更新場景判斷」？

子題 E：Primitive 判定問題——什麼是 Primitive、什麼是 Composite 的判定標準是什麼？在 DSPy 語境中，Primitive 對應 `dspy.ChainOfThought(single_signature)`，Composite 對應 `dspy.Module(forward=多個子呼叫)`——但第六層的 `prim_generate_gain()` 是確定性 Primitive，它既不是 Signature 也不是 Module。

盲點揭示

你可能忽略了一個關鍵問題：**DSPy 的型別系統不支援不確定性**。DSPy Signature 的輸入輸出型別是 `str`、`int`、`float`、`list`、`Pydantic model`——都是精確型別。但我們的語義 IR 需要每個值都帶 `confidence` 和 `alternatives`。解決方案是在 Pydantic 型別中嵌入 confidence 欄位，然後讓 GEPA 的 feedback metric 檢查 confidence 的合理性。

另一個盲點：**GEPA 的 per-predictor feedback 機制** 恰好對應我們「每個 Primitive 有獨立可靠性規格」的需求。GEPA 呼叫 metric 時會傳入 `pred_name`（正在優化的 predictor 名稱），這讓我們可以對 `describe_noise` 檢查「SNR > 20dB 時噪音不應該被判為 severe」，對 `gen_beam` 檢查「波束寬度不能小於 20°」——每個 Primitive 有自己的物理約束。

重新定義

這個問題更應該被看作是：**「怎麼用 DSPy 的 Module 組合語義來表達一個同時包含確定性計算和統計性推理的控制迴路，並用 GEPA 的文字回饋機制把領域物理知識注入 prompt 優化過程？」**

第二步：六人對談——圍著實作的設計審查

場景：白板上投影著跑成功的 demo 輸出和完整程式碼

陳教授（盯著前三層的純 Python 函式）

> 「我注意到你做了一個非常重要的設計決策：前三層完全不碰 DSPy。`prim_fft()`、`prim_estimate_snr()`、`comp_extract_full_features()` 都是純 Python + numpy 函式——它們沒有 `dspy.Signature`，沒有 LLM 呼叫，沒有任何可以被 GEPA 優化的參數。」
>
> 「**這是對的。** 因為 FFT 的正確性不需要優化——它由數學定義保證。如果你把 FFT 包裝成 `dspy.Predict('signal -> spectrum')`，GEPA 會試圖優化 FFT 的 prompt——這是荒謬的。你不優化牛頓定律的 prompt。」
>
> 「但這引出一個微妙的問題：`comp_extract_full_features()` 的輸出最終要被序列化成字串傳給第四層的 LLM Signature。你在程式碼裡把 `AcousticFeatures` 格式化成——」

python

```
features_str = (
    f"SNR: {acoustic_features.snr_db} dB, "
    f"RT60: {acoustic_features.rt60_s} s, "
    f"Active sources: {acoustic_features.n_active_sources}, ..."
)
```

> 「**這個序列化就是 Semantic Linker 的具體實現。** 它把確定性的 Pydantic 型別翻譯成 LLM 能理解的自然語言。而且——這個序列化的格式本身會影響第四層的準確率。如果你把 SNR 寫成 `'SNR: 1.7 dB'` vs `'the signal-to-noise ratio is approximately 1.7 decibels'`，LLM 的反應可能不同。」
>
> 「**這代表 Semantic Linker 本身也是一個可優化的組件——** 但你目前把它寫死在 `FullPerceptualDescription.forward()` 裡。未來的改進可能是把序列化格式也變成 GEPA 可優化的參數。」

王博士後（盯著 DSPy Signature 的設計）

> 「讓我理清你的映射。你建立了一個非常清晰的對應：」

```
語義 ISA 概念            →  DSPy 實現
──────────────────────────────────────────
Primitive 指令宣告        →  dspy.Signature（class-based）
Primitive 指令實現        →  dspy.ChainOfThought(Signature)
Composite 指令            →  dspy.Module with forward()
確定性 Primitive          →  純 Python 函式（不用 DSPy）
IR 型別系統               →  Pydantic BaseModel
可靠性規格                →  GEPA feedback metric 中的檢查邏輯
物理約束                  →  feedback 中的文字回饋（PHYSICS VIOLATION: ...）
Harness                   →  AcousticSemanticHarness(dspy.Module)
Compilation               →  dspy.GEPA.compile()
```

> 「我特別欣賞你處理 `GenerateFullStrategy` 的方式——它在同一個 `forward()` 裡混合了三種 BACKEND：」

python

```python
class GenerateFullStrategy(dspy.Module):
    def forward(self, ...):
        # LLM + physics constraint
        beam_result = self.gen_beam(...)       # dspy.ChainOfThought
        # LLM
        nr_result = self.gen_nr(...)           # dspy.ChainOfThought
        # deterministic — 純函式呼叫，不經過 LLM
        gain_result = prim_generate_gain_params(audiogram_json, scene_str)
```

> 「**這就是 DSPy 的設計哲學真正發光的地方：** `forward()` 方法裡可以寫任意 Python 程式碼，不限於 LLM 呼叫。DSPy Module 不是 LangChain 的 chain——它是 PyTorch Module 的精神繼承者。你可以在裡面放 numpy 計算、if/else 分支、迴圈、外部 API 呼叫。**GEPA 只會優化裡面有 `dspy.Predict` 或 `dspy.ChainOfThought` 的部分，純函式呼叫不受影響。**」
>
> 「這完美地解決了子題 B——確定性-統計性混合問題。確定性部分就是普通 Python，統計性部分用 DSPy Module 包裝。兩者在同一個 `forward()` 裡共存。」

Priya（系統架構師，盯著 Harness 的實現）

> 「讓我分析 `AcousticSemanticHarness` 怎麼實現 Harness 的四個子系統。」
>
> 「**Semantic Scheduler** 的實現特別巧妙——你用了 `dspy.context(lm=...)` 作為 context manager：」

python

```python
# 第四層用小模型（低延遲）
with dspy.context(lm=self.fast_lm):
    percept = self.perceptual_desc(...)

# 第五層用大模型（強推理）
with dspy.context(lm=self.strong_lm):
    scene = self.scene_understanding(...)
```

> 「**這就是 Scheduler 的全部——** DSPy 的 `dspy.context` 讓你在不同的程式碼區塊切換 LM。第四層的三個 Primitive（describe_noise, describe_speech, describe_env）都用 gpt-4o-mini，因為它們是相對簡單的「特徵→描述」翻譯。第五層的 reason_about_scene 用 gpt-4o，因為它需要跨維度推理。」
>
> 「**Semantic Runtime** 的 Error Recovery 用 try/except + fallback：」

python

```
try:
    percept = self.perceptual_desc(...)
except Exception as e:
    # LLM 超時或幻覺 → 返回保守的 fallback
    percept = dspy.Prediction(
        noise_description='[{"type":"unknown",...}]',
        confidence=0.3  # 低信心度觸發保守策略
    )
```

> 「**Persistent Store** 用 Python 實例變數：」

python

```
self.scene_history: list[str] = []
self.feedback_history: list[str] = []
self.current_preferences = {...}
self.current_dsp_params: Optional[DSPParameterSet] = None
```

> 「在真實系統中這些會持久化到資料庫，但概念是一樣的——跨 `forward()` 呼叫保持狀態。」
>
> 「最後 **Semantic Linker** 的實現散布在 `forward()` 的各個銜接處——每當上一層的輸出需要變成下一層的輸入時，就有一段序列化程式碼。現在它是硬寫的字串格式化，但理論上可以抽象成獨立的 Linker 模組，甚至讓 GEPA 優化序列化格式。」

張博士生（盯著 GEPA feedback metric）

> 「這是我覺得整份程式碼中最深刻的部分——`create_acoustic_feedback_metric()`。」
>
> 「一般的 GEPA metric 很簡單：score = 0 或 1，feedback = 'correct' 或 'wrong'。但你的 metric 做了三件特殊的事。」
>
> 「第一：**Per-predictor 回饋**。 你利用了 GEPA 提供的 `pred_name` 參數，對每個 Primitive 寫了獨立的檢查邏輯：」

python

```
if pred_name and "describe_noise" in pred_name:
    # 第四層噪音描述的可靠性檢查
    if gold.snr_db > 20 and any(s['severity'] == 'severe' for s in sources):
        feedback_parts.append(
            "PHYSICS VIOLATION: SNR > 20dB but noise rated severe."
        )
```

> 「**這就是 IR 中 `RELIABILITY: source_identification_accuracy >= 0.80` 的實際實現**。 可靠性規格不是一個抽象的文件——它是一段被 GEPA 反思 LM 讀取的文字回饋。GEPA 的反思 LM 看到 `PHYSICS VIOLATION: SNR > 20dB but noise rated severe` 後，會推理出：『啊，當 SNR 高的時候，我不應該把噪音評為 severe』，然後修改 `describe_noise` 的 prompt 加入這個約束。」
>
> 「第二：**物理約束以文字形式注入**。 看這段：」

python

```python
if pred_name and "gen_beam" in pred_name:
    if bw < 20:
        feedback_parts.append(
            "PHYSICS VIOLATION: beam_width < 20°. "
            "A 2-mic BTE array with 10mm spacing cannot form beams "
            "narrower than ~20° at speech frequencies."
        )
```

> 「**GEPA 的反思 LM 讀到這段後，不只知道 20° 是下限，還知道為什麼——** 因為兩麥克風 BTE 陣列的物理限制。下次生成 beam_width 時，它會在 prompt 裡加入這個物理知識。**這就是『物理約束通過 GEPA 文字回饋注入 prompt 優化過程』的具體實現。**」
>
> 「第三：**使用者偏好一致性檢查。** 在降噪 metric 裡：」

python

```python
if gold.user_prefs_natural and agg > 0.8:
    feedback_parts.append(
        "User prefers 'natural' sound but aggressiveness > 0.8."
    )
```

> 「這不是物理約束，而是使用者體驗約束。GEPA 同時學到物理知識和使用者偏好——兩者都以文字回饋的形式注入同一個優化迴路。」
>
> 「**對我的 cross-sensor 研究，這意味著：** 我可以在 feedback metric 裡寫 `'PHYSICS VIOLATION: DT model predicted LDV phase response that violates microphone-LDV transfer function at 5kHz'`——讓 GEPA 的反思 LM 學會物理約束，然後自動改進 prompt 來避免違反它們。」

產品經理（盯著第七層和偏好更新）

> 「讓我走過李伯伯按『不滿意』按鈕的完整路徑：」

python

```python
# 使用者輸入
user_action = "button_press:dissatisfied"

# 第七層 Primitive: 意圖解析
intent = self.parse_intent(
    user_action="button_press:dissatisfied",
    current_scene="李伯伯在菜市場魚攤...",
    user_history='{"noise_tolerance":"medium",...}'
)
# → primary_goal: "希望聽得更清楚面前的人在說什麼"

# 第七層 Primitive: 偏好更新
pref_update = self.update_prefs(
    current_preferences='{"noise_tolerance":"medium",...}',
    user_feedback="button_press:dissatisfied",
    current_scene="菜市場，多人交談，SNR 3dB",
    feedback_history="..."
```

```
)
# → updated_preferences: {"noise_tolerance":"low", ...}
# → drift_detected: false
```

> 「注意 `update_preferences` 的輸出有一個 `drift_detected` 布林值。如果系統發現李伯伯最近一個月不斷把降噪調高——以前 medium 就夠，現在要 low——它會設 `drift_detected = True`，暗示可能是聽力在退化，建議回診。**這是一個語義層的判斷——純數值系統無法做出『使用者的偏好變化模式暗示聽力退化』這種推理。**」
>
> 「而且整個偏好更新邏輯都在 GEPA 優化範圍內。如果 GEPA 的反思 LM 發現 `update_preferences` 老是在使用者按一次『不滿意』後就把 noise_tolerance 從 medium 跳到 low（太激進），它會修改 prompt 加入『偏好更新應該是漸進的，一次回饋只應微調 0.1 個等級』。」

陳教授（最終總結——看著完整架構圖）

> 「讓我把 DSPy + GEPA 的實現跟我們之前的 LLVM 類比對齊：」

```
LLVM 概念                DSPy + GEPA 實現                 狀態
──────────────────────────────────────────────────────────────
Source Language     →  7 層 Signature 定義              ✅ 已實現
LLVM IR             →  Pydantic 型別 + dspy.Prediction   ✅ 已實現
                       （但缺不確定性分布的型別系統）      ⚠️ 部分
Type System         →  Pydantic BaseModel + Enum          ✅ 已實現
                       （但缺 Semantic Equivalence）      ❌ 未實現
Frontend            →  Signature docstring → prompt       ✅ DSPy 自動
Optimization Pass →  GEPA.compile()                       ✅ 已實現
Backend             →  DSPy Adapter (ChatAdapter等)       ✅ DSPy 自動
Machine Code        →  最終 prompt token 序列             ✅ DSPy 自動
Hardware            →  LLM (GPT-4o / GPT-4o-mini)         ✅ 已實現
Runtime             →  try/except + fallback              ✅ 已實現
Linker              →  forward() 中的字串序列化           ⚠️ 硬寫的
ABI                 →  Signature 的 InputField/OutputField ⚠️ 缺語義等價
Scheduler           →  dspy.context(lm=fast/strong)       ✅ 已實現
Persistent Store    →  Python 實例變數                    ⚠️ 未持久化
Standard Library    →  prim_generate_gain_params()        ⚠️ 只有一個
Debug / Trace       →  dspy.inspect_history()             ✅ DSPy 內建
JIT                 →  ❌                                 ❌ 未實現
Package Manager     →  ❌                                 ❌ 未實現
```

> 「**重要的觀察：已實現的部分已經比你想像的多。** DSPy 自動處理了 Frontend（Signature → prompt）、Backend（prompt → API call）、Debug Trace（inspect_history）。GEPA 處理了 Optimization Pass。你需要手動建構的是：型別系統、Linker、Scheduler、Persistent Store。」
>
> 「但最關鍵的——**這份程式碼跑起來了。** 前三層產生了真實的聲學特徵（SNR: 1.7 dB, RT60: 0.5s），第六層的確定性增益計算產生了真實的 NAL-NL2 增益值。一旦你配好 API key，第四到七層也會跑起來。然後 GEPA 會開始優化每個 Primitive 的 prompt——物理約束和聽力學知識會通過 feedback metric 自動注入。」
>
> 「**這就是為什麼 DSPy + GEPA 是正確的技術選擇：** 它讓你寫 **程式**（不是 prompt），程式裡混合了確定性計算和統計性推理，而 GEPA 自動把領域知識編譯成更好的 prompt。你不再需要手動調 prompt——你需要做的是定義 Signature（指令宣告）、寫 Module（指令組合）、寫 feedback metric（可靠性規格）。**三件事都是結構化的、可版本控制的、可測試的 Python 程式碼。**」